

Vibe Coding Log - AttritionIQ

AttritionIQ was built 7-9 October 2026, with every change merged into main through a pull request. Full record with every entry: docs/VIBE_CODING_LOG_FULL.md.

Platforms

- **Claude CLI (agent):** project planning and environment setup; ideation research and sprint/phase planning through the Product Manager and Project Manager skills and sub-agents. The modules themselves, such as the RAG copilot, were built either with skills or by sub-agents.
- **Claude Desktop:** finding the best-optimised prompts.
- **Google AI Studio:** front-end finishing assets, and the API key for the Gemini brief.
- **GitHub:** one branch per change, pull requests into main.
- **Streamlit Community Cloud:** hosting.

Workflow

For each phase:

1. We gave the AI the rubric and the current results.
2. The AI drafted a plan whose decision rules and pass/fail checks were written down *before* any run (docs/PHASE3_PLAN.md, PHASE5_PLAN.md, PHASE6_PLAN.md).
3. We reviewed the plan.
4. The AI implemented and ran it.
5. We verified the result ourselves, then merged it through a pull request.

The AI did the work, but a person stepped in between every stage:

- **Before building:** we approved each plan and its rules; nothing was built without a reviewed plan.
- **During the work:** we answered the AI's questions on scope and chose between the options it laid out.
- **After each run:** we checked the results against the rules we had set, and sent work back when they did not hold.
- **Before anything went live:** we ran the app ourselves and merged every pull request by hand; the AI never committed or pushed on its own.

Prompt strategies that worked

- **We set the AI up like a team member:** we gave it a role (project adviser and product manager), a project description, a fixed workflow (plan, then framework, then build and test) and strict guardrails (plan before building, never commit or push on its own).
- **Sources for every number:** the AI was told to always cite its source when it presented data, and to label anything without one as a team assumption. The HR handbook cites EU/UK law and GitLab's public handbook.
- We gave the rubric first, then asked for the design, then the code. Asking for the design first avoided rework.
- We fixed each selection rule before seeing results (e.g. "simplest model within 1 std wins"), so the AI could not tune the story to the numbers.
- We asked for every number in a write-up to be checked against reports/metrics.json by script, not by reading.
- We asked for an honest baseline or ceiling for every claim (naive forecast, item-mean, noise-free oracle).

Log by phase

Phase	What the AI did	What we changed or rejected	How we verified
0-1 Setup, EDA	Repo skeleton, data checks, feature ablation	Asked for a paired, two-model ablation; all 4 engineered features dropped	pytest; ablation numbers reproduced
2 Risk model	4-model comparison, nested CV, calibration, threshold, SHAP, fairness	Rules in code before the full run; rejected dropping income to "fix" SHAP signs (costs PR-AUC, $p = 0.045$)	Threshold checked against the break-even formula; numbers checked by script
3 Other tracks	Segments, recommender, forecast, RAG with a 33-question eval	Kept the naive forecast because ARIMA lost in rolling CV; rejected SHAP clusters	Oracle ceiling for CF; JOLTS checked against FRED
4 Front-end	4 task-based pages, shared assumptions panel	Restyled Streamlit instead of a separate JS front-end, so everything stays in Python	Headless run of every page and interaction
5 Freeze	Fresh-clone reproducibility, app tests	Froze the README-only run so graders can reproduce it	Fresh clone: 25 tests passed; planted bug caught
6 Deploy	Host choice, model bundle, live checks, LLM timeout	Dropped Vercel and the custom domain; accepted the template brief when Gemini is overloaded	Live numbers match the frozen model; 26 tests passed

What the AI got wrong, and how we caught it

- **A result too good to be true:** the recommender scored better than a perfect benchmark, which is impossible. Two parts of the code were accidentally sharing the same random numbers, so the model was seeing the answers. We noticed because the result could not be real, fixed it and added a check.
- **An unfair comparison:** the AI picked the forecast settings using a score that cannot compare those options fairly. We changed the order of steps so the choice is valid.
- **Wrong age groups:** 50-year-olds were counted in the "40-49" group. We caught it when our own count of employees aged 50+ did not match.
- **The live site froze:** the brief waited forever for Gemini to reply and spun for over 4 minutes. We added a 30-second limit, after which the app shows a standard brief instead.

Decided by us, not the AI

Persona names (team vote), excluding sensitive attributes, keeping the default cost assumptions, Logistic Regression by the pre-set rule, TF-IDF on the host, and the free hosting tier.